GoBD-Prozessablauf

Lindenweg16 ERP – WooCommerce Plugin v0.8.0
Stand: 12.08.2026 · Vertraulich

Dieses Dokument beschreibt die Prozessabläufe des Lindenweg16 ERP Plugins im Hinblick auf die GoBD-Konformität (Grundsätze zur ordnungsmäßigen Führung und Aufbewahrung von Büchern, Aufzeichnungen und Unterlagen in elektronischer Form sowie zum Datenzugriff). Es dient als Ergänzung zur Verfahrensdokumentation und richtet sich an den Unternehmer, den Steuerberater und ggf. den Betriebsprüfer.

Dokumentierte Prozesse

Nr.	Prozess	Beschreibung
1	**Belegerfassung**	Manuelle und automatische Erfassung von Ausgabenbelegen
2	**Festschreibung**	Unveränderbar-Machen eines Belegs (GoBD-Schutz)
3	**Stornierung**	Gegenbuchung statt Löschung oder Änderung
4	**Rechnungserstellung**	Automatische Ausgangsrechnung bei Bestellabschluss
5	**E-Rechnungsempfang**	XRechnung/ZUGFeRD-Import und Belegübernahme
6	**Buchungsjournal**	Chronologisches, append-only Kassenbuch
7	**Hash-Kette (SHA-256)**	Manipulationsschutz über verkettete Hashes
8	**Betriebsprüfungs-Export**	Strukturierter Datenexport (Z3)
9	**GoBD-Integritätsprüfung**	Jährliche Verifikation aller Ketten und Nummern

1. Belegerfassung

Ausgabenbelege gelangen über drei Wege ins System. Jeder Beleg erhält eine lückenlose Belegnummer (Format **B-JJJJ-NNNN**) und wird sofort in die SHA-256-Hash-Kette eingehängt.

Erfassungswege

1	**Manuell (Foto-Upload)** Nutzer lädt Beleg-Foto hoch (JPEG/PNG/WebP), gibt Betrag, Datum, Händler, EÜR-Kategorie ein. Bild wird auf max. 2000px verkleinert.
2	**Wareneingang (automatisch)** Bei Wareneingang wird ein Beleg mit Verknüpfung zum Einkauf erzeugt. Quelle = „wareneingang".
3	**E-Rechnung (XRechnung/ZUGFeRD)** Upload einer XML- oder PDF-Datei → automatische Datenextraktion → Beleg-Übernahme auf Knopfdruck. Siehe Prozess 5.

Technischer Ablauf

1	**Belegnummer zuweisen** BelegRepository::next_nummer($jahr) → nächste freie Laufnummer. Format: B-JJJJ-NNNN, lückenlos pro Kalenderjahr.
2	**Datei sicher speichern** Upload in geschütztes Verzeichnis (wp-content/uploads/lw16-belege/), .htaccess-Schutz, MIME-Typ-Prüfung, Verkleinerung.
3	**Hash berechnen & einfügen** HashChain::prepare_insert('belege', $data) → SHA-256 aus Belegnummer, Datei, Betrag, Datum, Händler, Kategorie, Quelle, Zeitstempel + vorheriger Hash.
4	**INSERT in lw16_belege** Datensatz mit hash und vorheriger_hash in die Datenbank. Status: festgeschrieben = 0 (offen).

■ **Zeitgerechte Erfassung:** Die GoBD fordert die zeitnahe Erfassung. Die Prüffunktion warnt, wenn zwischen Belegdatum und Erfassungszeitpunkt mehr als 10 Tage liegen.

Technische Details

Tabelle:	wp_lw16_belege
Nummernformat:	B-JJJJ-NNNN (z. B. B-2026-0042)
Hash-Felder:	belegnummer, datei_pfad, betrag, datum, haendler, eur_kategorie, quelle, erstellt_am, erstellt_von
Nicht im Hash:	notiz (änderbar nach Festschreibung), storniert, storno_grund, id, hash

2. Festschreibung (Unveränderbarkeit)

Die Festschreibung macht einen Beleg unveränderbar – ein Kernprinzip der GoBD. Nach der Festschreibung sind nur noch Notiz-Änderungen und Stornierungen möglich, keine inhaltlichen Änderungen an Betrag, Datum, Händler oder Kategorie.

1	**Nutzer klickt „Festschreiben"** Bestätigungsdialog: „Beleg festschreiben? Danach nicht mehr änderbar."
2	**Berechtigungsprüfung** current_user_can(manage_lw16) + Nonce-Prüfung (CSRF-Schutz).
3	**Status setzen** festgeschrieben = 1, festgeschrieben_am = Systemzeitstempel. Diese Felder sind nicht im Hash (nachträglich gesetzte Metadaten).
4	**Buchung erzeugen** BuchungRepository::create_from_beleg() → Buchungssatz im Journal (Typ „ausgabe", Buchungsnummer BU-JJJJ-NNNN).
5	**Audit-Log** AuditLog::record('beleg_festgeschrieben') → wer, wann, welcher Betrag. Ebenfalls mit Hash-Kette geschützt.

■ **Unveränderbarkeit:** BelegRepository::update() prüft den festgeschrieben-Status. Bei festgeschrieben = 1 werden nur Änderungen am Feld „notiz" akzeptiert. Alle anderen Felder werden verworfen. Da „notiz" nicht im Hash ist, bricht eine Notiz-Änderung die Hash-Kette nicht.

3. Stornierung (Gegenbuchung)

Die GoBD verbietet das Löschen oder nachträgliche Ändern von festgeschriebenen Buchungen. Korrekturen erfolgen ausschließlich über Stornobuchungen (Gegenbuchungen). Der Originalbeleg bleibt erhalten und wird als „storniert" markiert.

Beleg-Storno

1	**Nutzer klickt „Stornieren"** Nur bei festgeschriebenen Belegen möglich. Storno-Belege können nicht nochmal storniert werden (Schutz vor Endlosschleife).
2	**Original markieren** storniert = 1, storno_grund = Begründung auf dem Originalbeleg. Felder storniert/storno_grund sind nicht im Hash → keine Kettenunterbrechung.
3	**Storno-Beleg anlegen** Neuer Beleg mit quelle = „storno", storno_von_id = Original-ID, gleicher Betrag und Kategorie. Eigene Belegnummer, eigener Hash. Sofort als festgeschrieben markiert (unveränderbar).
4	**Buchung stornieren** Zugehörige Buchung im Journal wird als storniert markiert. Neue Gegenbuchung (Typ „storno") im Journal mit eigenem Hash.
5	**Audit-Log** beleg_storniert: Grund, Original-Belegnummer, Storno-Belegnummer.

Rechnungs-Storno

1	**Rechnung als storniert markieren** storniert = 1, storno_grund auf dem Original. Es wird keine Storno-Rechnung erzeugt (Rechnung ist kein Kassenbuch-Eintrag).
2	**Buchung stornieren** Zugehörige Einnahme-Buchung wird storniert, Gegenbuchung erzeugt.
3	**Audit-Log** rechnung_storniert: Rechnungsnummer, Grund.

■ **EÜR-Auswirkung:** sum_by_category() schließt sowohl stornierte Belege (storniert = 1) als auch Storno-Belege (quelle = „storno") aus. Netto-Effekt: der stornierte Betrag wird nicht mehr gezählt.

4. Rechnungserstellung

Für jede abgeschlossene WooCommerce-Bestellung wird automatisch eine PDF-Rechnung mit lückenloser Rechnungsnummer erstellt. Die Erzeugung läuft asynchron über den Action Scheduler.

1	**Bestellstatus → „abgeschlossen"** WooCommerce-Hook woocommerce_order_status_completed feuert.
2	**Asynchrone Einreihung** Scheduler::enqueue_async() → Action Scheduler Job. Rechnung wird nicht im Kunden-Request erzeugt.
3	**Rechnungsnummer vergeben** Modus „jahr": JJJJ-NNNN (z. B. 2026-0001). Modus „durchlaufend": RE-NNNNN (z. B. RE-00042). Lückenlos.
4	**Dateiname VOR Insert generieren** Kritisch: datei_pfad ist im Hash. Daher wird der Dateiname vor dem INSERT festgelegt, damit der Hash korrekt berechnet wird.
5	**Hash berechnen & INSERT** HashChain::prepare_insert('rechnungen', $data) → Insert mit hash und vorheriger_hash.
6	**PDF erzeugen & speichern** PDF mit CI-Design (Logo, Tabelle, §19-Hinweis, IBAN). Gespeichert in wp-content/uploads/lw16-dokumente/.
7	**Buchung & Audit-Log** BuchungRepository::create_from_rechnung() → Einnahme-Buchung. AuditLog::record('rechnung_erstellt').

Technische Details

Tabelle:	wp_lw16_rechnungen
Nummernformat:	JJJJ-NNNN oder RE-NNNNN (konfigurierbar)
Hash-Felder:	order_id, rechnungsnummer, jahr, laufnummer, datei_pfad, betrag, erstellt_am
Speicherort:	wp-content/uploads/lw16-dokumente/ (.htaccess-geschützt)
Zugriff:	Authentifiziert: Manager oder Bestell-Eigentümer

5. E-Rechnungsempfang

Seit dem 1.1.2025 müssen alle inländischen Unternehmen im B2B-Bereich strukturierte E-Rechnungen empfangen können. Das System unterstützt XRechnung (XML, CII/UBL) und ZUGFeRD/Factur-X (PDF mit eingebetteter XML).

1	**Upload (XML oder PDF)** Datei-Upload über Admin-Oberfläche. Erlaubte Formate: .xml, .pdf. Datei wird in lw16-dokumente/ gesichert.
2	**Format erkennen** XML → XRechnung. PDF → ZUGFeRD-Extraktion (sucht <?xml-Block im PDF-Stream, prüft auf CII/UBL-Indikatoren).
3	**XML parsen** Automatische Erkennung: CII (CrossIndustryInvoice) oder UBL. Extrahiert: Rechnungsnummer, Datum, Betrag, Absendername, Steuer-ID.
4	**In Datenbank speichern** INSERT in lw16_erechnung_eingang mit Hash-Kette (HashChain::prepare_insert). Status = „empfangen".
5	**Als Beleg übernehmen (optional)** Nutzer klickt „Als Beleg übernehmen" → BelegRepository::create() mit Daten aus E-Rechnung. Status → „verarbeitet".

Technische Details

Tabelle:	wp_lw16_erechnung_eingang
Hash-Felder:	datei_pfad, format, absender_name, absender_steuer_id, rechnungsnummer_ext, betrag, datum, erstellt_am
Nicht im Hash:	status, beleg_id (ändern sich bei Verarbeitung), xml_inhalt
Formate:	XRechnung (CII, UBL), ZUGFeRD/Factur-X

6. Buchungsjournal

Das Buchungsjournal ist das zentrale, chronologische Kassenbuch. Es folgt strikt dem Append-only-Prinzip: keine UPDATEs, keine DELETEs. Korrekturen laufen über Storno-Gegenbuchungen.

Buchungstypen

Typ	Auslöser	Beschreibung
einnahme	Rechnung erstellt	WooCommerce-Bestellung abgeschlossen
ausgabe	Beleg festgeschrieben	Betriebsausgabe (manuell/Wareneingang/E-Rechnung)
storno	Stornierung	Gegenbuchung zu einer stornierten Buchung

Technische Details

Tabelle:	wp_lw16_buchungen
Nummernformat:	BU-JJJJ-NNNN (z. B. BU-2026-0001)
Hash-Felder:	buchungsnummer, buchungsdatum, erfasst_am, typ, betrag, kategorie, beschreibung, referenz_typ, referenz_id, erstellt_von
Timestamps:	buchungsdatum (wirtschaftlich) vs. erfasst_am (System) — getrennt gemäß GoBD

7. SHA-256-Hash-Kette

Jeder Datensatz in den steuerrelevanten Tabellen enthält einen SHA-256-Hash aus seinem Inhalt und dem Hash des Vorgänger-Datensatzes. Nachträgliche Manipulation bricht die Kette sichtbar – vergleichbar mit einer Blockchain.

Hash-Formel

```
hash = SHA-256( tabelle | feld1 | feld2 | ... | feldN | vorheriger_hash )
```

Der erste Datensatz einer Tabelle verwendet `GENESIS` als vorheriger_hash. Felder werden mit `|` getrennt.

Geschützte Tabellen

Tabelle	Hash-Felder	Ausgeschlossene Felder
belege	belegnummer, datei_pfad, betrag, datum, haendler, eur_kategorie, quelle, erstellt_am, erstellt_von	notiz, storniert, storno_grund
rechnungen	order_id, rechnungsnummer, jahr, laufnummer, datei_pfad, betrag, erstellt_am	storniert, storno_grund
buchungen	buchungsnummer, buchungsdatum, erfasst_am, typ, betrag, kategorie, beschreibung, referenz_typ, referenz_id, erstellt_von	storniert
audit_log	user_id, aktion, objekt_typ, objekt_id, details, erstellt_am	—
erechnung_eingang	datei_pfad, format, absender_name, absender_steuer_id, rechnungsnummer_ext, betrag, datum, erstellt_am	status, beleg_id, xml_inhalt

■ **Design-Prinzip:** Felder, die sich nach der Erstellung ändern dürfen (z. B. storniert, notiz, status), werden NICHT in den Hash einbezogen. So kann ein Beleg storniert oder eine Notiz geändert werden, ohne die Hash-Kette zu brechen.

8. Export für Betriebsprüfung

Der strukturierte Datenexport gemäß §147 Abs. 6 AO (Z3: Datenträgerüberlassung) stellt dem Betriebsprüfer alle steuerrelevanten Daten in maschinenlesbarem Format zur Verfügung.

1	**Zeitraum wählen** Nutzer wählt Start- und Enddatum (Standard: aktuelles Kalenderjahr).
2	**ZIP-Archiv erzeugen** Temporäre ZIP-Datei im Uploads-Verzeichnis.
3	**CSV-Dateien generieren** UTF-8 mit BOM, Semikolon-getrennt, Anführungszeichen als Textbegrenzer. Komma als Dezimaltrennzeichen. Kompatibel mit Excel und IDEA.
4	**Strukturbeschreibung** BESCHREIBUNG.txt (menschenlesbar) + INDEX.xml (maschinenlesbar, GDPdU-ähnlich).
5	**Download & Audit-Log** ZIP wird ausgeliefert, temporäre Datei gelöscht. AuditLog::record('bp_export') protokolliert den Export.

Enthaltene Dateien

Datei	Inhalt
belege.csv	Alle Ausgabenbelege (mit Hash, Storno-Status)
rechnungen.csv	Alle Ausgangsrechnungen
buchungen.csv	Buchungsjournal (Einnahmen, Ausgaben, Stornos)
audit_log.csv	Vollständiges Änderungsprotokoll
euer.csv	EÜR-Zusammenfassung (Einnahmen, Ausgaben, Gewinn)
BESCHREIBUNG.txt	Erklärung für den Prüfer (Format, Felder, Integritätsschutz)
INDEX.xml	Maschinenlesbare Strukturbeschreibung (GDPdU)

9. GoBD-Integritätsprüfung

Die Prüffunktion verifiziert die Integrität aller steuerrelevanten Daten. Empfehlung: mindestens jährlich vor dem Jahresabschluss. Ergebnisse werden in der Tabelle lw16_gobd_pruefung protokolliert.

1	**Prüfung starten** Nutzer wählt Kalenderjahr und klickt „Prüfung starten".
2	**Hash-Ketten verifizieren (5 Tabellen)** Für jede Tabelle: alle Datensätze in ID-Reihenfolge durchlaufen, Hash neu berechnen und mit gespeichertem Hash vergleichen. Vorgänger-Hash-Verkettung prüfen. Datensätze ohne Hash (vor Migration) werden übersprungen.
3	**Nummern-Lücken prüfen** Belegnummern (B-JJJJ-NNNN), Rechnungsnummern und Buchungsnummern (BU-JJJJ-NNNN) auf Lückenlosigkeit prüfen.
4	**Zeitgerechte Erfassung prüfen** Belege mit mehr als 10 Tagen Differenz zwischen Belegdatum und Erfassungszeitpunkt identifizieren (Warnung, kein Fehler).
5	**Ergebnis speichern** Status (ok/warnung/fehler), Fehleranzahl, Warnungsanzahl, Details (JSON) in lw16_gobd_pruefung. AuditLog::record('gobd_pruefung').

Durchgeführte Prüfungen

Prüfung	Typ	Bei Verstoß
Hash-Kette Belege	Integrität	Fehler
Hash-Kette Rechnungen	Integrität	Fehler
Hash-Kette Buchungen	Integrität	Fehler
Hash-Kette Audit-Log	Integrität	Fehler
Hash-Kette E-Rechnungen	Integrität	Fehler
Belegnummern lückenlos	Vollständigkeit	Fehler
Rechnungsnummern lückenlos	Vollständigkeit	Fehler
Buchungsnummern lückenlos	Vollständigkeit	Fehler
Zeitgerechte Erfassung	Ordnungsmäßigkeit	Warnung

Anhang: Datenmodell-Übersicht

Alle steuerrelevanten Tabellen mit ihren Hash-geschützten Spalten, Nummernkreisen und Beziehungen.

Tabelle	Nummernkreis	Hash	Append-only	Storno
lw16_belege	B-JJJJ-NNNN	✓	✓ (nach Festschr.)	Gegenbuchung
lw16_rechnungen	JJJJ-NNNN / RE-NNNNN	✓	✓	Markierung
lw16_buchungen	BU-JJJJ-NNNN	✓	✓	Gegenbuchung
lw16_audit_log	—	✓	✓	—
lw16_erechnung_eingang	—	✓	✓	—
lw16_gobd_pruefung	—	—	✓	—

Aufbewahrungsfristen

- **Rechnungen:** 10 Jahre (§14b UStG)
- **Buchungsbelege:** 8 Jahre (§147 AO n.F.)
- **Audit-Log:** Gesamte Aufbewahrungsdauer der zugehörigen Belege
- Fristbeginn: Schluss des Kalenderjahres der letzten Eintragung

GoBD-Prinzipien und ihre Umsetzung

GoBD-Grundsatz	Umsetzung im Plugin
Nachvollziehbarkeit	Audit-Log mit Hash-Kette, Benutzer-ID, Zeitstempel
Vollständigkeit	Lückenlose Nummernkreise, automatische Prüfung
Richtigkeit	Datenvalidierung, Berechtigungsprüfung, CSRF-Schutz
Zeitgerechte Erfassung	Getrennte Timestamps (Belegdatum vs. Systemzeit), 10-Tage-Prüfung
Ordnung	Strukturierte Kategorisierung, EÜR-Zuordnung
Unveränderbarkeit	Festschreibung, Hash-Ketten, Append-only, Storno statt DELETE

GoBD-Prozessablauf

Lindenweg16 ERP – WooCommerce Plugin v0.8.0
Stand: 12.08.2026 · Vertraulich

Dieses Dokument beschreibt die Prozessabläufe des Lindenweg16 ERP Plugins im Hinblick auf die GoBD-Konformität (Grundsätze zur ordnungsmäßigen Führung und Aufbewahrung von Büchern, Aufzeichnungen und Unterlagen in elektronischer Form sowie zum Datenzugriff). Es dient als Ergänzung zur Verfahrensdokumentation und richtet sich an den Unternehmer, den Steuerberater und ggf. den Betriebsprüfer.

Dokumentierte Prozesse

Nr.	Prozess	Beschreibung
1	**Belegerfassung**	Manuelle und automatische Erfassung von Ausgabenbelegen
2	**Festschreibung**	Unveränderbar-Machen eines Belegs (GoBD-Schutz)
3	**Stornierung**	Gegenbuchung statt Löschung oder Änderung
4	**Rechnungserstellung**	Automatische Ausgangsrechnung bei Bestellabschluss
5	**E-Rechnungsempfang**	XRechnung/ZUGFeRD-Import und Belegübernahme
6	**Buchungsjournal**	Chronologisches, append-only Kassenbuch
7	**Hash-Kette (SHA-256)**	Manipulationsschutz über verkettete Hashes
8	**Betriebsprüfungs-Export**	Strukturierter Datenexport (Z3)
9	**GoBD-Integritätsprüfung**	Jährliche Verifikation aller Ketten und Nummern

1. Belegerfassung

Ausgabenbelege gelangen über drei Wege ins System. Jeder Beleg erhält eine lückenlose Belegnummer (Format **B-JJJJ-NNNN**) und wird sofort in die SHA-256-Hash-Kette eingehängt.

Erfassungswege

1	**Manuell (Foto-Upload)** Nutzer lädt Beleg-Foto hoch (JPEG/PNG/WebP), gibt Betrag, Datum, Händler, EÜR-Kategorie ein. Bild wird auf max. 2000px verkleinert.
2	**Wareneingang (automatisch)** Bei Wareneingang wird ein Beleg mit Verknüpfung zum Einkauf erzeugt. Quelle = „wareneingang".
3	**E-Rechnung (XRechnung/ZUGFeRD)** Upload einer XML- oder PDF-Datei → automatische Datenextraktion → Beleg-Übernahme auf Knopfdruck. Siehe Prozess 5.

Technischer Ablauf

1	**Belegnummer zuweisen** BelegRepository::next_nummer($jahr) → nächste freie Laufnummer. Format: B-JJJJ-NNNN, lückenlos pro Kalenderjahr.
2	**Datei sicher speichern** Upload in geschütztes Verzeichnis (wp-content/uploads/lw16-belege/), .htaccess-Schutz, MIME-Typ-Prüfung, Verkleinerung.
3	**Hash berechnen & einfügen** HashChain::prepare_insert('belege', $data) → SHA-256 aus Belegnummer, Datei, Betrag, Datum, Händler, Kategorie, Quelle, Zeitstempel + vorheriger Hash.
4	**INSERT in lw16_belege** Datensatz mit hash und vorheriger_hash in die Datenbank. Status: festgeschrieben = 0 (offen).

■ **Zeitgerechte Erfassung:** Die GoBD fordert die zeitnahe Erfassung. Die Prüffunktion warnt, wenn zwischen Belegdatum und Erfassungszeitpunkt mehr als 10 Tage liegen.

Technische Details

Tabelle:	wp_lw16_belege
Nummernformat:	B-JJJJ-NNNN (z. B. B-2026-0042)
Hash-Felder:	belegnummer, datei_pfad, betrag, datum, haendler, eur_kategorie, quelle, erstellt_am, erstellt_von
Nicht im Hash:	notiz (änderbar nach Festschreibung), storniert, storno_grund, id, hash

2. Festschreibung (Unveränderbarkeit)

Die Festschreibung macht einen Beleg unveränderbar – ein Kernprinzip der GoBD. Nach der Festschreibung sind nur noch Notiz-Änderungen und Stornierungen möglich, keine inhaltlichen Änderungen an Betrag, Datum, Händler oder Kategorie.

1	**Nutzer klickt „Festschreiben"** Bestätigungsdialog: „Beleg festschreiben? Danach nicht mehr änderbar."
2	**Berechtigungsprüfung** current_user_can(manage_lw16) + Nonce-Prüfung (CSRF-Schutz).
3	**Status setzen** festgeschrieben = 1, festgeschrieben_am = Systemzeitstempel. Diese Felder sind nicht im Hash (nachträglich gesetzte Metadaten).
4	**Buchung erzeugen** BuchungRepository::create_from_beleg() → Buchungssatz im Journal (Typ „ausgabe", Buchungsnummer BU-JJJJ-NNNN).
5	**Audit-Log** AuditLog::record('beleg_festgeschrieben') → wer, wann, welcher Betrag. Ebenfalls mit Hash-Kette geschützt.

■ **Unveränderbarkeit:** BelegRepository::update() prüft den festgeschrieben-Status. Bei festgeschrieben = 1 werden nur Änderungen am Feld „notiz" akzeptiert. Alle anderen Felder werden verworfen. Da „notiz" nicht im Hash ist, bricht eine Notiz-Änderung die Hash-Kette nicht.

3. Stornierung (Gegenbuchung)

Die GoBD verbietet das Löschen oder nachträgliche Ändern von festgeschriebenen Buchungen. Korrekturen erfolgen ausschließlich über Stornobuchungen (Gegenbuchungen). Der Originalbeleg bleibt erhalten und wird als „storniert" markiert.

Beleg-Storno

1	**Nutzer klickt „Stornieren"** Nur bei festgeschriebenen Belegen möglich. Storno-Belege können nicht nochmal storniert werden (Schutz vor Endlosschleife).
2	**Original markieren** storniert = 1, storno_grund = Begründung auf dem Originalbeleg. Felder storniert/storno_grund sind nicht im Hash → keine Kettenunterbrechung.
3	**Storno-Beleg anlegen** Neuer Beleg mit quelle = „storno", storno_von_id = Original-ID, gleicher Betrag und Kategorie. Eigene Belegnummer, eigener Hash. Sofort als festgeschrieben markiert (unveränderbar).
4	**Buchung stornieren** Zugehörige Buchung im Journal wird als storniert markiert. Neue Gegenbuchung (Typ „storno") im Journal mit eigenem Hash.
5	**Audit-Log** beleg_storniert: Grund, Original-Belegnummer, Storno-Belegnummer.

Rechnungs-Storno

1	**Rechnung als storniert markieren** storniert = 1, storno_grund auf dem Original. Es wird keine Storno-Rechnung erzeugt (Rechnung ist kein Kassenbuch-Eintrag).
2	**Buchung stornieren** Zugehörige Einnahme-Buchung wird storniert, Gegenbuchung erzeugt.
3	**Audit-Log** rechnung_storniert: Rechnungsnummer, Grund.

■ **EÜR-Auswirkung:** sum_by_category() schließt sowohl stornierte Belege (storniert = 1) als auch Storno-Belege (quelle = „storno") aus. Netto-Effekt: der stornierte Betrag wird nicht mehr gezählt.

4. Rechnungserstellung

Für jede abgeschlossene WooCommerce-Bestellung wird automatisch eine PDF-Rechnung mit lückenloser Rechnungsnummer erstellt. Die Erzeugung läuft asynchron über den Action Scheduler.

1	**Bestellstatus → „abgeschlossen"** WooCommerce-Hook woocommerce_order_status_completed feuert.
2	**Asynchrone Einreihung** Scheduler::enqueue_async() → Action Scheduler Job. Rechnung wird nicht im Kunden-Request erzeugt.
3	**Rechnungsnummer vergeben** Modus „jahr": JJJJ-NNNN (z. B. 2026-0001). Modus „durchlaufend": RE-NNNNN (z. B. RE-00042). Lückenlos.
4	**Dateiname VOR Insert generieren** Kritisch: datei_pfad ist im Hash. Daher wird der Dateiname vor dem INSERT festgelegt, damit der Hash korrekt berechnet wird.
5	**Hash berechnen & INSERT** HashChain::prepare_insert('rechnungen', $data) → Insert mit hash und vorheriger_hash.
6	**PDF erzeugen & speichern** PDF mit CI-Design (Logo, Tabelle, §19-Hinweis, IBAN). Gespeichert in wp-content/uploads/lw16-dokumente/.
7	**Buchung & Audit-Log** BuchungRepository::create_from_rechnung() → Einnahme-Buchung. AuditLog::record('rechnung_erstellt').

Technische Details

Tabelle:	wp_lw16_rechnungen
Nummernformat:	JJJJ-NNNN oder RE-NNNNN (konfigurierbar)
Hash-Felder:	order_id, rechnungsnummer, jahr, laufnummer, datei_pfad, betrag, erstellt_am
Speicherort:	wp-content/uploads/lw16-dokumente/ (.htaccess-geschützt)
Zugriff:	Authentifiziert: Manager oder Bestell-Eigentümer

5. E-Rechnungsempfang

Seit dem 1.1.2025 müssen alle inländischen Unternehmen im B2B-Bereich strukturierte E-Rechnungen empfangen können. Das System unterstützt XRechnung (XML, CII/UBL) und ZUGFeRD/Factur-X (PDF mit eingebetteter XML).

1	**Upload (XML oder PDF)** Datei-Upload über Admin-Oberfläche. Erlaubte Formate: .xml, .pdf. Datei wird in lw16-dokumente/ gesichert.
2	**Format erkennen** XML → XRechnung. PDF → ZUGFeRD-Extraktion (sucht <?xml-Block im PDF-Stream, prüft auf CII/UBL-Indikatoren).
3	**XML parsen** Automatische Erkennung: CII (CrossIndustryInvoice) oder UBL. Extrahiert: Rechnungsnummer, Datum, Betrag, Absendername, Steuer-ID.
4	**In Datenbank speichern** INSERT in lw16_erechnung_eingang mit Hash-Kette (HashChain::prepare_insert). Status = „empfangen".
5	**Als Beleg übernehmen (optional)** Nutzer klickt „Als Beleg übernehmen" → BelegRepository::create() mit Daten aus E-Rechnung. Status → „verarbeitet".

Technische Details

Tabelle:	wp_lw16_erechnung_eingang
Hash-Felder:	datei_pfad, format, absender_name, absender_steuer_id, rechnungsnummer_ext, betrag, datum, erstellt_am
Nicht im Hash:	status, beleg_id (ändern sich bei Verarbeitung), xml_inhalt
Formate:	XRechnung (CII, UBL), ZUGFeRD/Factur-X

6. Buchungsjournal

Das Buchungsjournal ist das zentrale, chronologische Kassenbuch. Es folgt strikt dem Append-only-Prinzip: keine UPDATEs, keine DELETEs. Korrekturen laufen über Storno-Gegenbuchungen.

Buchungstypen

Typ	Auslöser	Beschreibung
einnahme	Rechnung erstellt	WooCommerce-Bestellung abgeschlossen
ausgabe	Beleg festgeschrieben	Betriebsausgabe (manuell/Wareneingang/E-Rechnung)
storno	Stornierung	Gegenbuchung zu einer stornierten Buchung

Technische Details

Tabelle:	wp_lw16_buchungen
Nummernformat:	BU-JJJJ-NNNN (z. B. BU-2026-0001)
Hash-Felder:	buchungsnummer, buchungsdatum, erfasst_am, typ, betrag, kategorie, beschreibung, referenz_typ, referenz_id, erstellt_von
Timestamps:	buchungsdatum (wirtschaftlich) vs. erfasst_am (System) — getrennt gemäß GoBD

7. SHA-256-Hash-Kette

Jeder Datensatz in den steuerrelevanten Tabellen enthält einen SHA-256-Hash aus seinem Inhalt und dem Hash des Vorgänger-Datensatzes. Nachträgliche Manipulation bricht die Kette sichtbar – vergleichbar mit einer Blockchain.

Hash-Formel

```
hash = SHA-256( tabelle | feld1 | feld2 | ... | feldN | vorheriger_hash )
```

Der erste Datensatz einer Tabelle verwendet `GENESIS` als vorheriger_hash. Felder werden mit `|` getrennt.

Geschützte Tabellen

Tabelle	Hash-Felder	Ausgeschlossene Felder
belege	belegnummer, datei_pfad, betrag, datum, haendler, eur_kategorie, quelle, erstellt_am, erstellt_von	notiz, storniert, storno_grund
rechnungen	order_id, rechnungsnummer, jahr, laufnummer, datei_pfad, betrag, erstellt_am	storniert, storno_grund
buchungen	buchungsnummer, buchungsdatum, erfasst_am, typ, betrag, kategorie, beschreibung, referenz_typ, referenz_id, erstellt_von	storniert
audit_log	user_id, aktion, objekt_typ, objekt_id, details, erstellt_am	—
erechnung_eingang	datei_pfad, format, absender_name, absender_steuer_id, rechnungsnummer_ext, betrag, datum, erstellt_am	status, beleg_id, xml_inhalt

■ **Design-Prinzip:** Felder, die sich nach der Erstellung ändern dürfen (z. B. storniert, notiz, status), werden NICHT in den Hash einbezogen. So kann ein Beleg storniert oder eine Notiz geändert werden, ohne die Hash-Kette zu brechen.

8. Export für Betriebsprüfung

Der strukturierte Datenexport gemäß §147 Abs. 6 AO (Z3: Datenträgerüberlassung) stellt dem Betriebsprüfer alle steuerrelevanten Daten in maschinenlesbarem Format zur Verfügung.

1	**Zeitraum wählen** Nutzer wählt Start- und Enddatum (Standard: aktuelles Kalenderjahr).
2	**ZIP-Archiv erzeugen** Temporäre ZIP-Datei im Uploads-Verzeichnis.
3	**CSV-Dateien generieren** UTF-8 mit BOM, Semikolon-getrennt, Anführungszeichen als Textbegrenzer. Komma als Dezimaltrennzeichen. Kompatibel mit Excel und IDEA.
4	**Strukturbeschreibung** BESCHREIBUNG.txt (menschenlesbar) + INDEX.xml (maschinenlesbar, GDPdU-ähnlich).
5	**Download & Audit-Log** ZIP wird ausgeliefert, temporäre Datei gelöscht. AuditLog::record('bp_export') protokolliert den Export.

Enthaltene Dateien

Datei	Inhalt
belege.csv	Alle Ausgabenbelege (mit Hash, Storno-Status)
rechnungen.csv	Alle Ausgangsrechnungen
buchungen.csv	Buchungsjournal (Einnahmen, Ausgaben, Stornos)
audit_log.csv	Vollständiges Änderungsprotokoll
euer.csv	EÜR-Zusammenfassung (Einnahmen, Ausgaben, Gewinn)
BESCHREIBUNG.txt	Erklärung für den Prüfer (Format, Felder, Integritätsschutz)
INDEX.xml	Maschinenlesbare Strukturbeschreibung (GDPdU)

9. GoBD-Integritätsprüfung

Die Prüffunktion verifiziert die Integrität aller steuerrelevanten Daten. Empfehlung: mindestens jährlich vor dem Jahresabschluss. Ergebnisse werden in der Tabelle lw16_gobd_pruefung protokolliert.

1	**Prüfung starten** Nutzer wählt Kalenderjahr und klickt „Prüfung starten".
2	**Hash-Ketten verifizieren (5 Tabellen)** Für jede Tabelle: alle Datensätze in ID-Reihenfolge durchlaufen, Hash neu berechnen und mit gespeichertem Hash vergleichen. Vorgänger-Hash-Verkettung prüfen. Datensätze ohne Hash (vor Migration) werden übersprungen.
3	**Nummern-Lücken prüfen** Belegnummern (B-JJJJ-NNNN), Rechnungsnummern und Buchungsnummern (BU-JJJJ-NNNN) auf Lückenlosigkeit prüfen.
4	**Zeitgerechte Erfassung prüfen** Belege mit mehr als 10 Tagen Differenz zwischen Belegdatum und Erfassungszeitpunkt identifizieren (Warnung, kein Fehler).
5	**Ergebnis speichern** Status (ok/warnung/fehler), Fehleranzahl, Warnungsanzahl, Details (JSON) in lw16_gobd_pruefung. AuditLog::record('gobd_pruefung').

Durchgeführte Prüfungen

Prüfung	Typ	Bei Verstoß
Hash-Kette Belege	Integrität	Fehler
Hash-Kette Rechnungen	Integrität	Fehler
Hash-Kette Buchungen	Integrität	Fehler
Hash-Kette Audit-Log	Integrität	Fehler
Hash-Kette E-Rechnungen	Integrität	Fehler
Belegnummern lückenlos	Vollständigkeit	Fehler
Rechnungsnummern lückenlos	Vollständigkeit	Fehler
Buchungsnummern lückenlos	Vollständigkeit	Fehler
Zeitgerechte Erfassung	Ordnungsmäßigkeit	Warnung

Anhang: Datenmodell-Übersicht

Alle steuerrelevanten Tabellen mit ihren Hash-geschützten Spalten, Nummernkreisen und Beziehungen.

Tabelle	Nummernkreis	Hash	Append-only	Storno
lw16_belege	B-JJJJ-NNNN	✓	✓ (nach Festschr.)	Gegenbuchung
lw16_rechnungen	JJJJ-NNNN / RE-NNNNN	✓	✓	Markierung
lw16_buchungen	BU-JJJJ-NNNN	✓	✓	Gegenbuchung
lw16_audit_log	—	✓	✓	—
lw16_erechnung_eingang	—	✓	✓	—
lw16_gobd_pruefung	—	—	✓	—

Aufbewahrungsfristen

- **Rechnungen:** 10 Jahre (§14b UStG)
- **Buchungsbelege:** 8 Jahre (§147 AO n.F.)
- **Audit-Log:** Gesamte Aufbewahrungsdauer der zugehörigen Belege
- Fristbeginn: Schluss des Kalenderjahres der letzten Eintragung

GoBD-Prinzipien und ihre Umsetzung

GoBD-Grundsatz	Umsetzung im Plugin
Nachvollziehbarkeit	Audit-Log mit Hash-Kette, Benutzer-ID, Zeitstempel
Vollständigkeit	Lückenlose Nummernkreise, automatische Prüfung
Richtigkeit	Datenvalidierung, Berechtigungsprüfung, CSRF-Schutz
Zeitgerechte Erfassung	Getrennte Timestamps (Belegdatum vs. Systemzeit), 10-Tage-Prüfung
Ordnung	Strukturierte Kategorisierung, EÜR-Zuordnung
Unveränderbarkeit	Festschreibung, Hash-Ketten, Append-only, Storno statt DELETE

Generiert am 12.08.2026 aus Lindenweg16 ERP v0.8.0. Dieses Dokument ersetzt keine steuerliche Beratung.